Project description: Bear in hat

My idea was originally to create a maze, but realizing that I don't understand why I don't create walls normally, I decided to make this game.

The meaning of the game is controlling the player (bear) by the arrows on the keyboard to collect hats-earflaps. Each collected cap earns a point. You have 30 seconds to collect as many caps as you can.

Each cap appears and disappears randomly. At the end of the game, there is an opportunity to view your account and close the game yourself. The score starts at 0 every game. The invoice is recorded and stored in a text file - invoice. During the game, you can try to write down your name, it will be visible in the upper-right corner. The bear does not have the opportunity to appear behind the screen, as well as hats. The bear has the ability to move diagonally. The screen displays the timer and the score during the game. At the end of the game, everything disappears from the screen and you can only see your score.

I was inspired by this project, so I have a desire to continue it further. At the moment, the code meets all the criteria, so I hand it over in this (in fact, for a full-fledged game -terrible) form.

My updates will include:

1. adding a home screen with the ability to enter a name, start a game, exit the game, turn off or turn on music, see the top 3
2. Adding music
3. pause screen in the game, with buttons-exit, continue or on/off music
4. Top 3 best players in a text file.
5. When you score 15 points , the speed increases twice
6. The new coin - honey- accelerates 3 times for 4 seconds.
7. Opponents- while I'm thinking about it, you can make the difficulty more difficult (also choose it in the menu) - so opponents can drop out instead of coins, collecting which either decreases the speed or minus points

I have zip file, including:

1. Code of game .py
2. Requirements.txt
3. Icon of game
4. Sprite-bear
5. Sprite-hat
6. This pdf file with description
7. score.txt

Example use of the project requirements:

• Comparisons and logical operations:

```
if event.type == pygame.QUIT:
```

```
 if remaining_time == 0:
         running= False
```

```
if keys[pygame.K_RIGHT] and hero_x+hero_width<SCREEN_WIDTH :
```

```
        new_hero_x += hero_speed
```

```
score+=1
            current_score += 1
```

- **if-else statements**

```
if coin_rect.colliderect(hero_rect):
            #coin dissapears if we touch it
            coins.remove(coin)
            # score -> +coin = +score
            score+=1
            current_score += 1
            #new coin in list to show up new coin
            coins.append(coin_display())
```

and also a million more if in my code

I didn't add else, since it doesn't matter. But if you are not sure of my knowledge, then else in the example above could be :
else

score = score

current_score =current_ score

- **lists and at least 2 different list manipulations (append, pop, indexing, etc)**

```
            coins.append(coin_display())

coins = [coin_display()]

 coins.remove(coin)
```

list+ 2 manipulations

- **for and/or while loops**

```
for coin in coins:
        coin_x, coin_y = coin
        # to have no error in colliderrect
        coin_rect = pygame.Rect(coin_x, coin_y, 72, 60)
        # new coin appears
        screen.blit(coin_image,(coin_x, coin_y))
```

```
while gameover:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            gameover = False
```

```
while running:

    # view each event in turn from the list of all events
    for event in pygame.event.get():
```

```
        #text input by user
        if event.type == pygame.TEXTINPUT:
            input_text+=event.text
```

- **use input(), print() and typecasting**

```
input_text=""
input_text_font = pygame.font.SysFont("Arial", 30)
def input_text_display(text, font, text_col, x,y):
    # img is text with font and color
    input_text_img = font.render(text, True, text_col)
    # show that text (img) on position xy
    screen.blit(input_text_img, (x,y))
```

```
   #text input by user
        if event.type == pygame.TEXTINPUT:
            input_text+=event.text
```

```
    input_text_display("Username: "+input_text, input_text_font,(0,0,0), 1000,10)
```

```
def text_display(text, font, text_col, x,y):
    # img is text with font and color
    img = font.render(text, True, text_col)
    # show that text (img) on position xy
    screen.blit(img, (x,y))
```

```
text_font = pygame.font.SysFont("Arial", 30)

text_display("Score: " + str(score), text_font, (0, 0, 0), 10, 10)
```

- **function implementations**

```
# Fill in the background color when the game is on
    fill_background()
    # Function about adding hero
    hero_image_display(hero_x, hero_y)
    # allows user to put name and there is output by criteria ( what he inputs, font, color,
location)
    input_text_display("Username: "+input_text, input_text_font,(0,0,0), 1000,10)
    # The display of text (Score, score to string, font, color, x,y)# current score reads from
file in the loop to be updated
    current_score = read_score()
    text_display("Score: " + str(score), text_font, (0, 0, 0), 10, 10)
write_score(current_score)

    fps_time()
```

```
# A function that fills the screen with the selected color
def fill_background():
    screen.fill(beige)
# hero image on display function
```

```
def hero_image_display(hero_x,hero_y):
    #add hero image to hero_image
    hero_image=pygame.image.load("hero.png")
    #hero add to display + coordinates
    screen.blit(hero_image,(hero_x,hero_y))
# hero  movement function
def hero_movement_display(hero_x,hero_y, keys):
    # Initialize new_x and new_y to the current position
    new_hero_x, new_hero_y = hero_x, hero_y

    # if press button, then happens movement; i use if not elif because of that way hero can
move diagonally; 'and' means that hero doesnt dissapear behind the
    if keys[pygame.K_RIGHT] and hero_x+hero_width<SCREEN_WIDTH :
        new_hero_x += hero_speed
    if keys[pygame.K_LEFT] and hero_x>0 :
        new_hero_x -= hero_speed
    if keys[pygame.K_DOWN] and hero_y+hero_height<SCREEN_HEIGHT :
        new_hero_y += hero_speed
    if keys[pygame.K_UP] and hero_y>0 :
        new_hero_y -= hero_speed

    return new_hero_x, new_hero_y
# function time
def fps_time():
    clock.tick(FPS)

#coin is image
coin_image=pygame.image.load("coin.png")
# function that makes coins in random place
def coin_display():
```

- **write and read from files**

```
def read_score():
    try:
        #if we have txt file score , then we read it and get int that is score
        with open('score.txt', 'r') as file:
            score = int(file.read())
            # if we cannot find the file
    except FileNotFoundError:
        # If the file doesn't exist, return a score of 0
        score = 0
    return score
#function that writes score in text file
def write_score(score):
    with open('score.txt', 'w') as file:
        file.write(str(score))
```

Acknowledgements I would like to thank my previous experience in game development on Unity with c# .

PS

At school, I got 4 out of 10 for a paygame project (4 years ago), because I didn't understand how to work with this module. Now I understand this, but units are still more convenient)) I figured out everything from and to, but it's harder)))))

I declar

"I declare that the work submitted here is from my authorship only. I haven't used any gen-

erative AI to help with any code/text included in my work. I have given credit for the help I had

conceptualizing my project. My work respects the university and course code of conduct"